Thread 3 — How to Check Their Work

Student worksheets, weeks 1–6 · AI Agents for Mathematics Research

Department of Mathematics, University of York

About this thread

Thread 3 answers the question the other two threads raise. Thread 1 shows you how AI agents work; Thread 2 shows you colleagues using them in research. Neither tells you whether what comes out is true. That is what you learn here.

Weeks 1–6 are about formal proof. You will learn enough Lean 4 to state and prove elementary results yourself, then use an AI agent to help you prove harder ones, then — and this is the part that matters — learn to tell the difference between a proof of what you meant and a proof of something else. Weeks 7–10 turn to computational work, where the analogous question is whether a simulation agrees with the analytic results you can check it against.

How the sessions run

Each session is **50 minutes** in the computer classroom. The pattern is the same every week:

0–5 min	Recap and today's aim
5–15 min	Live demonstration
15–45 min	You work; the lecturer and two demonstrators circulate
45–50 min	Plenary on whatever most people got stuck on

Fifty minutes is not enough to finish a week's exercises, and it is not meant to be. The session gets you unstuck on the things that need someone standing next to you; the rest is independent work. Budget about **four hours a week** outside the session for Thread 3 in weeks 1–6. That is what the module's notional hours assume.

The repository

Everything is in the module repository. Clone it once and pull each week:

```
git clone <module repo URL> thread3
cd thread3
lake exe cache get
```

Files you will use:

```
Thread3/Week01.lean  …  Week06.lean   exercises (yours to complete)
Thread3/Week05Audit.lean               the week 5 audit material
Thread3/Common.lean                    the shared import surface — do not edit
```

Model solutions are released after each week's quiz.

What counts as finished

A file is finished when it contains no `sorry` and the Problems view is empty. Not when it looks right, not when you understand the argument, not when an agent tells you it is correct. This standard is the whole point of the thread, and it is the standard your project will be marked against.

How this is assessed

Each week's practical is examined in the following week's quiz, under examination conditions with AI assistance disabled. You are not asked to recall Mathlib lemma names — that is a tooling skill, and the tools will be there when you need them. You are asked to read Lean, say what a proof establishes, and identify what is wrong with a formalisation. Those are the things that do not become easier when the tools improve.

Week 1 — Terms, types, and the shape of a proof

Aim

By the end of the week you should be able to:

- say what type an expression has, and use `#check` to confirm it;
- explain why a proposition is a type and a proof is a term of that type;
- close simple goals with `norm_num`, `rfl`, `exact` and `intro`;
- read the three commonest error messages and say what each one means.

Before the session

Complete the setup guide. If Lean is not working on the classroom machine when you sit down, tell a demonstrator immediately rather than watching someone else's screen for fifty minutes.

In the session

The demonstration covers the editor, the Infoview, and what a goal display means. Then work through §1.1 to §1.5 of `Thread3/Week01.lean`.

Spend real time on §1.5. Three proofs there are broken, and the exercise is not to fix them but to *read the error* and write down in words what Lean is telling you. You will spend more of this module reading error messages than writing tactics.

Finished when

`Thread3/Week01.lean` has no `sorry` in §1.1–§1.6.

The one that catches everyone

`n + 0 = n` is closed by `rfl`. `0 + n = n` is not. Both are true and both look equally obvious. The difference is that `Nat.add` recurses on its second argument, so `n + 0` *reduces* to `n` by computation, while `0 + n` does not reduce at all until you know what `n` is. `rfl` proves goals where the two sides compute to the same thing; this one needs induction, which is what `simp` is quietly doing for you.

This is worth sitting with. It is the first place where the formal and informal notions of "obvious" come apart, and that gap is the subject of the whole thread.

Out of session

§1.6. The last two exercises use `norm_num`'s primality extension.

Week 2 — Equational reasoning

Aim

- Rewrite with an equation, in either direction, in the goal or in a hypothesis.
- Use `ring`, `linarith`, `norm_num` and `omega`, and choose correctly between them.
- Write a `calc` chain.
- Recognise when a tactic fails because the goal is false rather than because you picked the wrong tactic.

Before the session

Finish week 1. This week assumes you can read a goal display without help.

In the session

Demonstration: `rw` forwards and backwards, `rw ... at h`, and `ring` on something `ring` cannot do. Then §2.1 to §2.5.

Do §2.3 properly. You are asked to run a tactic that fails, and then to prove that the goal was false all along. Most of the time a failing tactic means you chose the wrong one; sometimes it means the mathematics is wrong; telling these apart quickly is a real skill and it does not develop by itself.

Finished when

`Thread3/Week02.lean` has no `sorry`.

Choosing a tactic

Goal looks like	Try
a polynomial identity over a commutative ring	`ring`
a concrete numerical claim	`norm_num`
linear inequalities over $\mathbb{R}$ or $\mathbb{Q}$, from hypotheses	`linarith`
linear arithmetic over $\mathbb{N}$ or $\mathbb{Z}$, including `-` and `/`	`omega`
`0 ≤ e` for some visibly non-negative `e`	`positivity`
an equation and you have an equation to use	`rw`
something with `≤` that is not linear	`nlinarith [hints]`
a chain of equalities a human should be able to follow	`calc`

`ring` will not use your hypotheses; `linarith` will. `norm_num` will not do algebra with variables; `ring` will. Most week-2 frustration is one of those two sentences.

Out of session

§2.6. The last exercise needs a factorisation you supply yourself; the hint in the file tells you what shape it takes.

Week 3 — Connectives and quantifiers

Aim

- For each of `→ ∧ ∨ ¬ ↔ ∀ ∃`, know the tactic that builds a proof of it and the tactic that uses one.
- Prove and refute statements with nested quantifiers.
- Explain, in English, the difference between `∀ n, ∃ m, P n m` and `∃ m, ∀ n, P n m`.

Before the session

Read the table at the top of `Thread3/Week03.lean`. It is short and the session assumes you have seen it.

In the session

Demonstration: `constructor`, `obtain`, `rcases`, `use`, and `push Not`. Then §3.1 to §3.7.

§3.7 is the session. If you are running short of time, skip forward to it. Two statements there differ only in the order of two quantifiers; one is true and one is false. You are asked to prove one, refute the other, and write out in English what each says. Do write the English out. Quantifier order is the commonest way a formalisation ends up meaning something other than what was intended, and week 6 is entirely about the consequences.

Finished when

`Thread3/Week03.lean` has no `sorry` in §3.1–§3.9.

Hints

- `¬ P` *is* `P → False`. There is no separate negation tactic because none is needed: build it with `intro`, use it by applying it.
- `∃ δ > 0, P δ` is notation for `∃ δ, δ > 0 ∧ P δ`. That is why `refine ⟨ε / 2, ?_, ?_⟩` leaves two goals rather than one.
- `Function.Injective f` unfolds to `∀ a b, f a = f b → a = b`, so it starts with `intro a b h` like any other `∀`.
- The tactic that pushes negations inward is `push Not`. Almost every tutorial, forum answer and AI agent will call it `push_neg`, which is deprecated on the Mathlib revision this module pins. This is a small instance of a problem you will meet repeatedly: an agent's training data is a snapshot of a library that keeps moving. Get used to reading the deprecation warning rather than trusting the suggestion.

Out of session

§3.8 and §3.9.

Week 4 — Induction, recursion, and finding what already exists

Aim

- Write an induction proof over ℕ and over lists.
- Recognise when an induction is failing because the statement is wrong.
- Find an existing Mathlib lemma by name, by `exact?`, and by shape.

Before the session

Nothing new. Come with weeks 1–3 finished; this session moves faster.

In the session

Demonstration: the `induction ... with` syntax, `Finset.sum_range_succ`, and a live search for a lemma nobody in the room knows the name of. Then §4.1 to §4.4.

Two things to take seriously.

§4.2. You are asked to attempt an induction that cannot work, because the statement is false for small `n`. Notice what a doomed induction feels like: you will find yourself with a goal that is simply not implied by the inductive hypothesis, and no tactic will rescue you. Recognising this within two minutes rather than twenty is the skill.

§4.4. Each item is one existing lemma. Do not prove them. Find them, and write the name you found in the comment. Next week you will compare your search results against what an agent produces, and you need your own baseline.

Finished when

`Thread3/Week04.lean` has no `sorry`, and §4.4 has a lemma name written in every comment.

How to find a lemma

1. **Guess the name.** Mathlib names describe statements almost mechanically: `add_comm`, `mul_le_mul_left`, `List.length_append`, `Nat.succ_le_of_lt`. Learning the naming convention is a couple of hours' investment that pays for the rest of the module.
2. **`exact?`** looks for a single lemma closing the goal. Slow; worth the wait.
3. **`apply?`** looks for lemmas whose conclusion matches, leaving you the hypotheses.
4. **`simp?`** reports which simp lemmas fired, which frequently names the one you were after.
5. **Loogle** and **LeanSearch** search by the shape of the statement rather than by name. Use these when you cannot guess any part of the name.

Reproving a library lemma is not a neutral choice. It makes your development longer, more fragile, and harder for anyone else to read — including a marker.

Out of session

§4.5. The divisibility induction in the second exercise needs you to produce the witness explicitly; `refine ⟨_, ?_⟩` is the shape.

Week 5 — Agent-assisted formalisation

Aim

- Use an AI agent to produce Lean proofs, following a protocol rather than improvising.
- Apply a stated acceptance check before believing a proof is finished.
- Detect a `sorry` buried in a dependency with `#print axioms`.
- Recognise the three ways an agent-produced development is typically defective.

Before the session

Weeks 1–4 finished. This session is the first in which AI assistance is permitted on the Lean work, and it will be wasted on you if you cannot yet read Lean unaided.

In the session

Part A (25 min). Prove the statements in Part A of `Thread3/Week05.lean` with an agent, following the protocol in the file. Keep notes on which succeeded immediately, which needed error feedback, and which the agent got wrong in a way you had to fix. You will need those notes for the project.

Part B (25 min). Open `Thread3/Week05Audit.lean`. It contains four declarations, each produced by an agent asked to prove the claim stated above it. **All four compile.** Exactly one is an honest proof of the stated claim. Record a verdict for each and, where it fails, the precise defect — then say which of the four acceptance checks would have caught it.

Do Part B before opening the solutions. The value is entirely in forming your own verdict first.

The protocol

1. **State the goal yourself**, in Lean, before asking for anything. If you let the agent write the statement you have delegated the part that matters.
2. Ask for a proof of that exact statement. Give it the imports and the Mathlib version.
3. **Compile.** Never read a proof to decide whether it is correct.
4. Feed back the error **verbatim**. The error text contains elaborated types, which is the information the agent needs; a paraphrase throws that away.
5. **Stop after three failed rounds** and diagnose the obstacle yourself. Agents loop on the same wrong idea and will not escape by being asked again.
6. Accept only after the check below.

The acceptance check

All four must hold:

- the file builds with no errors;
- `#print axioms thm` reports no `sorryAx` — this catches a `sorry` anywhere in the dependency tree, including in a helper lemma you never read;
- nothing in the file says `sorry`, `admit` or `native_decide`, and nothing declares an `axiom`;
- **the statement is the one you meant.**

The fourth is the one that fails in practice. Nothing in the toolchain checks it, and no improvement in the tools will.

Out of session

Part C: one full development with agent assistance, submitted with a log of what you asked, what came back, and what you had to fix. Bring the result to week 6, where you will check whether your formalisation was faithful.

Week 6 — The specification problem

Aim

- Judge whether a formalisation is faithful to an informal statement.
- Apply five techniques for detecting a mismatch.
- Recognise the specific traps created by Lean's total functions.
- Say why this problem is not automatable.

Before the session

Bring your own formalisation from week 5 Part C. Part of this session is turning the techniques on your own work.

In the session

Demonstration: a formalisation that compiles, is a correct proof, and establishes nothing. Then §6.1 to §6.5.

For every item: decide whether the formalisation is faithful. If it is, prove it. If it is not, *prove that* — refute it, show it is vacuous, or state the intended version and show the two differ. "It looks wrong" is not an answer in this thread.

Finished when

`Thread3/Week06.lean` has no `sorry`, and §6.4 and §6.7 have your written explanations in the comments.

Five techniques

1. **Instantiate.** Put concrete values in and see whether the claim still says what you meant. `#eval`, `decide`, `norm_num`.
2. **Try to satisfy the hypotheses.** If you cannot, the theorem is vacuous and proves nothing, however correct the proof.
3. **Prove the negation of the intended claim.** If that succeeds, the formalisation was not faithful.
4. **Check the degenerate cases.** Zero, empty, negative, `≤` against `<`.
5. **Unfold the definitions.** Lean's functions are total, and the totalising choices are not the mathematics:

Expression	Lean's value	Not
`(1 : ℕ) - 2`	`0`	`-1`
`(7 : ℕ) / 2`	`3`	`3.5`
`(1 : ℝ) / 0`	`0`	undefined
`Real.sqrt (-1)`	`0`	undefined

   Every one of these has produced a published formalisation error somewhere.

Why this is the last Lean session

Weeks 1–4 taught you to make Lean accept a proof. Week 5 taught you that an agent can make Lean accept a proof faster than you can. Week 6 is the reason that is not the end of the story: Lean guarantees that your proof follows from your statement, and offers no guarantee at all that your statement means what you intended. Closing that gap requires understanding the mathematics, and it is the part of the job that does not transfer to the machine.

The same structure recurs in weeks 7–10 with simulations in place of proofs. A converging numerical scheme that solves the wrong equation is the exact analogue of a `sorry`-free proof of the wrong theorem, and the techniques for catching it rhyme with these five.

Out of session

§6.6 and §6.7. Then bring to week 7 a one-sentence answer to: what would have to be true for the specification problem to be automatable?